Units 3 & 4 Study Guide

Unsupervised Learning and Sequential Models
split evenly across a team of three

Team member	Register number	Focus
Sandhiya S	RA2411003010045	Clustering geometry & Markov chains
Nishanth Kannan	RA2411003010051	Mixtures, EM & hidden Markov models
Netti Nithin Chakravarthy	RA2411003010033	Latent dimensions & sequence decoding

What this document covers

All 17 topics from the Unit 3 and Unit 4 syllabi, divided so that each member carries a comparable load, owns material from BOTH units, and is responsible for one genuinely hard derivation. Each topic gives the core idea, the equations worth memorising, the points that earn marks, what our own project measured for that topic, and likely viva questions. Read your own section in full; read the other two sections' 'core idea' and 'equations' boxes so the team can answer across the whole syllabus.

Course: Machine Learning (21CSC305P) • Project: Explainable AI-Driven Decision Support for Predictive Inventory and Procurement Optimization in ERP Systems • Repository: github.com/sandhiya-s86/xai-inventory-procurement

How the syllabus is divided

Member	Owns	Unit 3 topics	Unit 4 topics	Hard derivation
Sandhiya S	Grouping, geometry and chains	Clustering and measuring dissimilarity K-means clustering and the elbow method Hierarchical clustering, linkage and dendrograms	Sequential data Markov models	Lance-Williams linkage update and the elbow rule
Nishanth Kannan	Mixtures, EM and hidden states	Mixtures of Gaussians The EM algorithm and the alternative view of EM	Hidden Markov Models Maximum likelihood and EM for HMMs (Baum–Welch) The forward–backward algorithm Scaling factors	The EM lower bound and the Baum-Welch E-step
Netti Nithin Chakravarthy	Latent dimensions and decoding	PCA and choosing the number of latent dimensions Factor analysis, varimax and identifiability Evaluating the output of clustering methods	The Viterbi algorithm The sum–product algorithm Linear dynamical systems and the Kalman filter	The Kalman gain and the Viterbi recursion

Two ideas that cross the whole syllabus

1. EM is one algorithm, not two. The Gaussian mixture in Unit 3 and Baum-Welch in Unit 4 are the same E-step/M-step structure over different latent variables — 'which component?' versus 'which state at time t?'. Whoever is asked about either should be able to draw the parallel.

2. Sum-product unifies the inference. Forward-backward is sum-product on a chain; Viterbi is max-product on the same chain; the Kalman filter is the same recursion with a continuous Gaussian state instead of a discrete one. Four algorithms, one idea.

Marks are usually lost on the same handful of points, so each member should be able to state all of these regardless of who owns the topic:

- Inertia always falls with k, so the lowest inertia is never the right k.
- K-means is HARD assignment; a GMM gives responsibilities.
- EM is monotonic but only LOCALLY optimal, and the bound is tight only at the E-step posterior.
- Factor loadings are identified only up to an orthogonal rotation.
- Never shuffle a time series.
- Every row of a transition matrix sums to 1.
- Forward = evaluation, Viterbi = decoding, Baum-Welch = learning.
- Filtering uses the past only; smoothing uses the future and is therefore inadmissible as a forecasting feature.
- Complexity is O(T N^2), not exponential in T.

Member 1 — Sandhiya S

RA2411003010045 • Grouping, geometry and chains

The distance-based half of Unit 3 and the observable half of Unit 4. You own the question 'what does it mean for two things to be similar, and what does it mean for yesterday to matter?'

Hard derivation you own: Lance-Williams linkage update and the elbow rule

Clustering and measuring dissimilarity [Unit 3]

Core idea. Clustering splits unlabelled data so that points in a group are as similar as possible and points in different groups as dissimilar as possible. The crucial thing to internalise is that a clustering algorithm never sees your data — it only ever sees the dissimilarity between pairs. The metric is therefore not a detail, it IS the modelling assumption.

```
Euclidean   d = sqrt( SUM (a_i - b_i)^2 )
Manhattan   d = SUM |a_i - b_i|
Cosine      d = 1 - (a.b) / (|a| |b|)
Correlation d = cosine distance after centring both vectors
```

- Euclidean compares **magnitude**: two items are close only if they sell at similar volume.
- Cosine compares **direction** only. A slow seller and a fast seller with the same weekly shape become neighbours, because length is discarded.
- Correlation is cosine after removing each vector's own mean: pure shape, with both level and scale gone.
- Scale matters as much as the metric. Mixing units/day (0-400) with a zero-rate (0-1) makes an un-standardised Euclidean distance almost entirely the demand columns. Always z-score first, using **training** statistics.
- Clustering types: hierarchical (tree), centroid-based (k-means), density-based (DBSCAN), distribution-based (GMM). The last three are collectively non-hierarchical.

In our project

Five metrics were compared on the 200-SKU profile matrix. Manhattan ranks pairs almost identically to Euclidean (Spearman rho = 0.978) but cosine does not (0.588) and correlation even less (0.488) — concrete proof that the metric changes the answer.

Likely viva questions

1. Why can't you compare two clusterings produced under different metrics by their inertia?
2. When would cosine distance be the RIGHT choice for inventory, and when would it be badly wrong?
3. What does a zero vector do to cosine distance, and how should the implementation handle it?

K-means clustering and the elbow method [Unit 3]

Core idea. Partition n points into k clusters so that within-cluster variance is minimised. Lloyd's algorithm alternates two steps, each of which is the exact minimiser of the objective holding the other fixed — which is why inertia can never increase, and why the algorithm must terminate.

```
J = SUM_i || x_i - mu_{c(i)} ||^2        (inertia)
assign:  c(i) = argmin_j || x_i - mu_j ||
update:  mu_j = mean of the points assigned to j
```

- Steps: choose k centroids, assign each point to the nearest, recompute each centroid as the mean of its members, repeat until no point changes cluster.
- **Hard assignment.** Every point belongs to exactly one cluster with no measure of confidence. This single limitation is the entire motivation for Gaussian mixtures.
- Converges to a **local** minimum only. Hence k-means++ seeding (each new centre drawn with probability proportional to squared distance from the nearest existing centre) and multiple restarts keeping the lowest inertia.
- Elbow method: inertia always falls as k rises and hits zero at k = n, so the minimum is never the answer. The elbow is where the improvement stops being worth an extra cluster.
- Make the elbow reproducible: take the point on the curve furthest from the straight chord joining its two ends, rather than eyeballing a chart.

> ***In our project***
>
> Implemented from scratch; inertia matches scikit-learn to 1e-6 and the partition agrees at ARI 1.000. Inertia was verified non-increasing across all 9 iterations. The elbow selects k = 5 with final inertia 1,448.

Likely viva questions

1. Prove that inertia cannot increase between iterations.
2. What happens if a cluster becomes empty, and what should the code do about it?
3. Why is k-means++ better than picking k points uniformly?
4. Why is the k with the lowest inertia never the answer?

Hierarchical clustering, linkage and dendrograms [Unit 3]

Core idea. Agglomerative clustering starts with every point as its own cluster and repeatedly merges the two closest, until one remains. The sequence of merges IS the model: a dendrogram whose height at each join is the distance at which those two groups became one. You never have to commit to k in advance; you cut the tree afterwards.

```
single    d(ij,k) = min( d_ik, d_jk )
complete  d(ij,k) = max( d_ik, d_jk )
average   d(ij,k) = (n_i d_ik + n_j d_jk) / (n_i + n_j)
ward      merges the pair that raises within-cluster
          variance the least
```

- Agglomerative is bottom-up; divisive is top-down (start with one cluster and split).
- All four linkages come from one **Lance-Williams** update, which gives the distance from a newly merged cluster to every other as a weighted combination of distances that already exist — so the matrix is never recomputed.
- **Single linkage chains.** Two genuinely separate groups get bridged by a line of intermediate points, producing one huge cluster and several singletons.
- Ward is closest in spirit to k-means and is usually the best default on standardised numeric data. It is only valid for Euclidean distance.
- **Cophenetic correlation** measures how faithfully the dendrogram reproduces the original pairwise distances. High fidelity is NOT the same as a useful partition.

In our project

All four linkages match SciPy exactly on merge heights and cluster sizes. A genuine tension emerged: average linkage scored the best cophenetic correlation (0.931) yet put **97% of SKUs in one cluster** — chaining. Ward scored worst (0.495) but was the only balanced partition, so the cut was taken from Ward. Be ready to defend that choice.

Likely viva questions

1. What is chaining and which linkage suffers from it most?
2. Why is Ward undefined for cosine distance?
3. A dendrogram has an inversion (a merge lower than the one below it). What does that tell you?
4. How do you read the number of clusters off a dendrogram?

Sequential data [Unit 4]

Core idea. Data whose ORDER carries information: the present is interpreted using the past. Shuffling destroys structure. The chain rule gives the exact joint distribution, and every sequential model is a simplification of the conditioning history on its right-hand side.

```
p(x_1:T) = p(x_1) PROD_{t=2..T} p(x_t | x_1:t-1)

rho(k) = Cov(X_t, X_t-k) / sqrt(Var(X_t) Var(X_t-k))

windowed predictor:  y_t = b + SUM_{k=1..L} w_k x_t-k

log-likelihood:  l(theta) = SUM_t log p(x_t | history)
```

- The simplifications: finite window (AR), state-space (Markov), recurrent memory. Each decides what counts as 'the past'.
- **Autocorrelation** measures how strongly a series resembles a lagged copy of itself. The 95% white-noise band is +/- 1.96/sqrt(n).
- **Stationarity** means distributional properties are stable over time. ADF tests for a unit root but is blind to variance drift — always check both.
- Sequence tasks: forecasting (predict x_t+1), sequence classification (map the whole sequence to a label), sequence labelling (one output per time step).
- **Never shuffle a time series.** Random splits put day t+1 in train and day t in test, leaking the answer. Use chronological or rolling-origin evaluation.
- Normalise using TRAINING statistics only. Handle missing values without peeking forward. State clearly what is known at time t.

In our project

Order was shown to carry information rather than assumed: 29 of 30 lags are significantly autocorrelated, and an AR(7) model scores **0.179 nats/day** higher than an i.i.d. model. ADF rejects a unit root (p = 0.0128) yet the block standard deviation still varies by 2.5x — the two readings disagree, and both are reported.

Likely viva questions

1. Why is k-fold cross-validation wrong for a time series?
2. What exactly does the chain rule say, and how does a Markov model simplify it?
3. ADF says stationary but the variance clearly drifts. What do you conclude?
4. What is rolling-origin evaluation?

Markov models [Unit 4]

Core idea. The Markov assumption compresses the entire history into the current state: the next state is conditionally independent of everything earlier given the present one. That turns an unbounded conditioning history into a small matrix.

```
p(s_t | s_1:t-1) = p(s_t | s_t-1)

A_ij = p(S_t = j | S_t-1 = i),  rows sum to 1

p(s_1:T) = pi_s1 PROD_t a_{s_t-1, s_t}

P(S_t+k = j | S_t = i) = (A^k)_ij

stationary:  pi* = pi* A,  SUM_i pi*_i = 1

MLE from counts:  a_ij = N_ij / SUM_k N_ik
```

- Every ROW of A is a probability distribution and sums to 1. Check this first, always.
- A^k gives k-step transitions. Use repeated squaring, which costs O(log k) matrix products.
- The stationary distribution is the LEFT eigenvector for eigenvalue 1. Solve it as a linear system with the normalisation replacing one redundant equation — more stable than eigendecomposition.
- State types: **absorbing** (a_ii = 1, can never leave), **recurrent** (return is certain), **transient** (may be left forever).
- Expected holding time in a state is 1/(1 - a_ii), a geometric mean.
- Use LOG probabilities for path probabilities — a 600-step product underflows. A zero transition makes the whole path impossible (-inf).
- A zero COUNT is a strong claim: it declares a transition impossible. Laplace smoothing turns 'never seen' into 'merely unlikely'.
- Higher-order chains can be converted to first-order by expanding the state.

In our project

The key subtlety: the four demand states are equal-sized quantiles, so the stationary distribution comes out uniform **by construction** and proves nothing. The real test is persistence lift against independence — up to 1.96x for PEAK — with chi-square 63.0, p = 3.5e-10. Busy days genuinely cluster.

Likely viva questions

1. State the first-order Markov property precisely.
2. How do you find a stationary distribution, and when does it exist uniquely?
3. A state has never been observed as a source. What should its row of A be, and why not zeros?
4. Why compute path probabilities in logs?

Member 2 — Nishanth Kannan

RA2411003010051 • Mixtures, EM and hidden states

The latent-variable engine shared by both units. EM appears in Unit 3 as the Gaussian mixture and in Unit 4 as Baum-Welch; they are the same algorithm, and you own that connection.

Hard derivation you own: The EM lower bound and the Baum-Welch E-step

Mixtures of Gaussians [Unit 3]

Core idea. A GMM is the SOFT version of k-means. Instead of one label per point, every point gets a responsibility r_nk — the posterior probability that component k generated it — and those responsibilities sum to 1. A point sitting between two clusters is reported as 0.55/0.45 instead of being forced into one box.

```
p(x) = SUM_k pi_k N(x | mu_k, Sigma_k),   SUM_k pi_k = 1

r_nk = pi_k N(x_n|mu_k,S_k) / SUM_j pi_j N(x_n|mu_j,S_j)

Each component: mean mu (centre), covariance Sigma (shape

and width), mixing weight pi (how large the component is)
```

- Distribution-based clustering: points in a cluster are assumed to come from the same distribution.
- Unlike k-means, a GMM has a **covariance** per component, so clusters can be elliptical and of different sizes rather than spherical and equal.
- Fitted by EM, not by a closed form, because the component that generated each point is a hidden variable.
- **Choosing K:** BIC = -2 log L + p log n. Unlike inertia, BIC has a real interior minimum because the penalty grows with the parameter count, so it can genuinely choose K.
- A d-by-d covariance has d(d+1)/2 free entries, not d^2 — it is symmetric. Get this right in a parameter count.
- **Failure mode:** a component capturing a single point collapses to zero variance and the likelihood diverges to +infinity. Real, not hypothetical; the fix is a small regulariser on the covariance diagonal.

In our project

BIC selected K = 5. The log-likelihood rose from -1,256 to -1,106 over 28 iterations. Crucially, **31 of 200 SKUs** had no component above 90% responsibility — k-means reports all of them as confident.

Likely viva questions

1. Why is a GMM better than k-means for an inventory decision?
2. Why does a full covariance need d(d+1)/2 parameters?
3. What happens if a component collapses onto one point?
4. Why is BIC usable for choosing K when inertia is not?

The EM algorithm and the alternative view of EM [Unit 3]

Core idea. EM estimates parameters when some variables are hidden. E-step: compute the posterior over the hidden variables using current parameters. M-step: re-estimate parameters by maximising the expected complete-data log-likelihood. The alternative view explains WHY it works, and is the part most often asked about.

```
E: q(z) = p(z | x, theta_old)

M: theta_new = argmax_theta E_q[ log p(x, z | theta) ]
```

```
F(q,theta) = E_q[log p(x,z|theta)] + H(q)
           = log p(x|theta) - KL( q || p(z|x,theta) )
```

- Read the second line carefully — it is the whole proof. Because KL ≥ 0, F is a **lower bound** on the log-likelihood.
- The E-step maximises F over q. The maximiser is exactly the posterior, at which KL = 0 and the bound becomes **tight**: F equals the log-likelihood.
- The M-step maximises F over theta with q fixed. So each step raises F, and the E-step makes F equal the log-likelihood — therefore the log-likelihood **cannot decrease**.
- EM converges to a **local** stationary point, not the global maximum. Initialisation matters; use multiple restarts.
- Key terms: latent variables (hidden structure), likelihood, log-likelihood (turns products into numerically safe sums), MLE, posterior / responsibility, convergence.
- **Soft assignments:** responsibilities lie strictly between 0 and 1, so expected counts are fractional. That is what makes it EM rather than counting hard assignments.

> ***In our project***
>
> The identity was verified numerically rather than asserted. At the true posterior the gap between the bound and the log-likelihood is **2.3e-13** (machine zero). At any other q the gap is strictly positive and equals the KL divergence exactly. Monotonicity was confirmed on every iteration of both the GMM and the HMM fits.

Likely viva questions

1. Prove that EM cannot decrease the observed-data log-likelihood.
2. When exactly is the EM lower bound tight, and why?
3. Why is the log of a sum hard to optimise directly?
4. What is the difference between E_q[log p(x,z)] and log p(x)?

Hidden Markov Models [Unit 4]

Core idea. An HMM combines an UNOBSERVED Markov state sequence with observed emissions generated from those hidden states. Use it when the thing driving the system cannot be measured directly — a demand regime, a spoken phoneme, a machine fault.

```
lambda = (pi, A, B)

p(z_1:T, x_1:T) = p(z_1) PROD p(z_t|z_t-1) PROD p(x_t|z_t)

pi_i   = p(z_1 = i)

a_ij   = p(z_t = j | z_t-1 = i)

b_j(x) = p(x_t = x | z_t = j)
```

- **Keep z and x conceptually separate.** z is the hidden state; x is what you observe. Confusing them is the most common error.
- Conditional independences: given z_t, x_t does not depend on anything else; given z_t-1, z_t is independent of z_1:t-2. These are exactly what make dynamic programming possible.
- **The three canonical problems** — know this table cold: Evaluation (p(x|lambda)) via Forward; Decoding (most likely state path) via Viterbi; Learning (estimate lambda) via Baum-Welch / EM.
- Emissions can be discrete (each state has a probability vector over symbols, each row summing to 1) or continuous (Gaussian with a mean and variance per state; mixtures for multimodal).
- The state posterior gamma_t(i) = P(z_t = i | x_1:T) is computed by forward-backward.

- EM returns states in arbitrary order, so 'state 0' means nothing across runs. Sort by emission mean to get stable labels.

> ***In our project***
>
> A 3-state Gaussian HMM on log1p demand found QUIET, NORMAL and SURGE regimes with self-transitions of 0.94-0.99, i.e. persisting for weeks. SURGE landed on the October-December peak in both years — without the model ever being told Christmas exists.

Likely viva questions

1. Write the HMM joint factorisation and name each term.
2. Name the three problems and the algorithm for each.
3. What conditional independences does an HMM assume?
4. Why use log1p demand rather than raw demand for Gaussian emissions?

Maximum likelihood and EM for HMMs (Baum–Welch) [Unit 4]

Core idea. Baum-Welch is EM applied to an HMM. The E-step runs forward-backward to get expected state occupancy and expected transition usage; the M-step just normalises those expected counts.

```
E-step:  gamma_t(i), xi_t(i,j)  from forward-backward

M-step:  pi_i  = gamma_1(i)

         a_ij  = SUM_t xi_t(i,j) / SUM_t gamma_t(i)

         b_j(k) = SUM_{t: x_t=k} gamma_t(j) / SUM_t gamma_t(j)

stop when |l(new) - l(old)| < eps
```

- **Expected (fractional) counts, not hard counts.** That is what makes it EM rather than counting a Viterbi path. Responsibilities lie between 0 and 1, so counts can be fractional.
- Monotonic in likelihood, but converges only to a LOCAL optimum and is sensitive to initialisation.
- **A symmetric start is a saddle point.** If all rows of A begin equal the states are exchangeable and nothing breaks the symmetry — EM crawls along the saddle and can look converged while the states have never differentiated. Fix with genuinely asymmetric random starts plus multiple restarts, and a minimum iteration count.
- Floor the emission variances: a state capturing one point otherwise collapses to zero variance and the likelihood diverges.
- The M-step for a Gaussian HMM is a responsibility-weighted mean and variance — structurally identical to the GMM M-step. Unit 3 and Unit 4 meet here.

> ***In our project***
>
> This saddle-point failure was hit for real during development: a near-uniform initialisation converged after 3 iterations to A entries all near 0.5 when the truth was 0.9/0.8. Fixing the initialisation and adding restarts recovered A to within 0.01 of truth. On the demand series, Baum-Welch converged in 44 iterations with the log-likelihood rising monotonically from -367 to -329.

Likely viva questions

1. Write the Baum-Welch re-estimation formula for a_ij.
2. Why must you use expected counts rather than hard counts?
3. Why is a uniform initialisation a bad idea for an HMM?
4. How is the GMM M-step related to the Gaussian HMM M-step?

The forward–backward algorithm [Unit 4]

Core idea. Computes sequence likelihood and smoothed state posteriors by combining evidence from the past (forward) and the future (backward), avoiding any enumeration of the exponentially many state paths.

```
alpha_1(i) = pi_i b_i(x_1)

alpha_t(j) = b_j(x_t) SUM_i alpha_t-1(i) a_ij

p(x_1:T)   = SUM_i alpha_T(i)

beta_T(i)  = 1

beta_t(i)  = SUM_j a_ij b_j(x_t+1) beta_t+1(j)

gamma_t(i) = alpha_t(i) beta_t(i) / SUM_j alpha_t(j) beta_t(j)

xi_t(i,j) PROPTO alpha_t(i) a_ij b_j(x_t+1) beta_t+1(j)
```

- alpha_t(i) is the JOINT probability of the observations up to t AND being in state i at t. beta_t(i) is the probability of the FUTURE observations given state i at t.
- **Complexity O(T N^2) in time and O(T N) in memory**, versus N^T for enumeration. This is the headline number.
- **Filtering vs smoothing** — a distinction with practical consequences: filtering is P(z_t | x_1:t), past and present only; smoothing is P(z_t | x_1:T) and uses the future too. Filtering is proportional to alpha; smoothing to alpha*beta.
- Only filtering is admissible for a real-time decision. Using a smoothed estimate as a forecasting feature is data leakage.
- xi supplies the expected transition counts for the EM M-step; sum_j xi_t(i,j) must equal gamma_t(i).
- Initialise alpha with pi times the first emission, and beta_T with 1. Normalise gamma at every time.

In our project

Verified against brute-force enumeration of all 3^7 = 2,187 paths on a small HMM: the posteriors match to 1e-15. On the real series, filtering and smoothing disagree on **134 of 604 days** (22.2%) — those are the days where future evidence changes the verdict. Mean confidence rises from 0.859 to 0.898.

Likely viva questions

1. Define alpha and beta precisely in words.
2. What is the complexity, and why is that better than enumeration?
3. What is the difference between filtering and smoothing, and when must you use filtering?
4. Show that sum_j xi_t(i,j) = gamma_t(i).

Scaling factors [Unit 4]

Core idea. The forward recursion multiplies a probability at every step. Over a long sequence the running product underflows float64 to exactly zero, silently turning the likelihood into -inf and every posterior into NaN. Scaling normalises at each step and keeps the normaliser.

```
c_t = SUM_i alpha~_t(i)

alpha_t(i) = alpha~_t(i) / c_t        so SUM_i alpha_t(i) = 1

log p(x_1:T) = SUM_t log c_t

beta_t(i) = [ SUM_j a_ij b_j(x_t+1) beta_t+1(j) ] / c_t+1

log-sum-exp:  log SUM e^u = m + log SUM e^(u - m)
```

- Why underflow happens: typical probabilities are below 1, so p^T shrinks exponentially. A mathematically nonzero likelihood becomes 0 numerically.
- After scaling, the alpha vector **sums to 1 at every t** — it is now exactly the filtered posterior. Good self-check.
- The likelihood is recovered as a SUM of logs, which cannot underflow.
- The backward pass must use the SAME constants c_t. Mixing conventions between scaled and unscaled formulas is the classic HMM implementation bug.
- Log-domain arithmetic with log-sum-exp is the alternative remedy, and is what Viterbi uses (max commutes with log, so no scaling is needed there at all).
- Always state your definition of c_t, and store every one.

In our project

Rather than claim an underflow, the real numbers were measured. At the true T = 604 the raw product is about 1e-217 — small but still ABOVE the 1e-308 floor, so this series does not actually underflow, and the scaled and unscaled answers agree to the last decimal. Tiling the series to find the breaking point: naive survives T = 900 and is dead by **T = 1000**, while scaled is still finite at 3600.

Likely viva questions

1. Why does the forward recursion underflow but Viterbi in logs does not?
2. Show that log p(x) = sum of log c_t.
3. What must the scaled alpha vector sum to, and why?
4. What goes wrong if you scale alpha but not beta?

Member 3 — Netti Nithin Chakravarthy

RA2411003010033 • Latent dimensions and decoding

The dimensionality half of Unit 3 and the inference half of Unit 4. You own 'how many hidden things are there, and which one was active?'

Hard derivation you own: The Kalman gain and the Viterbi recursion

PCA and choosing the number of latent dimensions [Unit 3]

Core idea. PCA finds the orthogonal directions of maximum variance. The first component is the single direction along which the data varies most; the second is the maximum-variance direction orthogonal to it, and so on. It is a rotation of the axes, nothing more.

```
X = U S V^T        (SVD of the CENTRED data)

components  = rows of V^T

eigenvalue_i = s_i^2 / (n - 1)

explained variance ratio_i = eig_i / SUM eig
```

- Compute by SVD of the centred matrix, not by eigendecomposing X^T X — forming X^T X squares the condition number and loses precision on correlated columns.
- Components are **orthonormal**: V V^T = I. A good self-check on any implementation.
- An eigenvector is only defined up to sign, so every library needs a convention (scikit-learn flips each component so its largest-magnitude loading is positive).
- **Three rules for how many dimensions:** Kaiser (keep eigenvalues > 1, meaningful only on standardised data where an average component has eigenvalue exactly 1); cumulative variance threshold (direct but arbitrary); scree elbow (largest drop between consecutive eigenvalues).
- They routinely disagree. Report all three rather than quoting whichever gives the number you wanted.
- PCA is also one of the factor **extraction** methods, alongside principal axis factoring and canonical factoring.

In our project

The three rules gave 5, 8 and 1 components on the SKU profile matrix. Kaiser was used (L = 5), capturing 73.2% of variance. The elbow rule returning 1 is a known weakness on a smoothly decaying spectrum — reported, not hidden.

Likely viva questions

1. Why SVD rather than eigendecomposition of the covariance?
2. Why does the Kaiser rule only make sense on standardised data?
3. What does it mean that explained variance ratios sum to 1?
4. How would you verify a PCA implementation is correct?

Factor analysis, varimax and identifiability [Unit 3]

Core idea. A probabilistic latent-variable model: a few CONTINUOUS hidden factors generate the many observed variables. Where a mixture model's hidden variable is a discrete 'which group?', factor analysis assumes each observation is influenced by continuous latent factors.

```
z ~ N(0, I),    x = W z + mu + eps,    eps ~ N(0, Psi)

cov(x) = W W^T + Psi      (low rank + DIAGONAL)
```

```
posterior:  Sigma_z = (I + W^T Psi^-1 W)^-1
            E[z|x]  = Sigma_z W^T Psi^-1 (x - mu)
```

- **FA vs PCA is a real distinction, not a naming quibble.** PCA explains TOTAL variance and has no noise model, so it will spend a component on a column that is pure noise. FA explains only SHARED variance, pushing column-specific variance into the diagonal Psi.
- Parameter efficiency: W costs O(Ld) and diagonal Psi costs O(d), against O(d^2) for a full covariance. The claim is that all correlation is generated by L shared factors.
- **Identifiability:** for any orthogonal R, W' = WR gives W'W'^T = WRR^TW^T = WW^T — an identical covariance and an identical likelihood. The loadings are NOT unique.
- There are L(L-1)/2 free rotations, which is why the free parameter count is d + Ld - L(L-1)/2.
- Non-identifiability affects **interpretation**, not predictive performance. It is also the licence under which varimax operates: rotating costs nothing in fit.
- **Varimax** maximises the variance of the squared loadings, pushing each towards 0 or +/-1 so each factor loads on few variables — 'simple structure'. Orthogonal rotations (varimax, quartimax) keep factors uncorrelated; oblique ones (promax, oblimin) allow correlation.
- Assumptions: linearity, multivariate normality, no severe multicollinearity, homoscedasticity, independent observations. Suitability checks: Bartlett's test and the KMO measure (> 0.6 acceptable).
- **Communality** is the share of a variable's variance explained by the factors; a low value means that column is mostly its own noise.

> ***In our project***
>
> Rotating the fitted loadings by a random orthogonal matrix moved individual loadings by up to **1.42** while the log-likelihood moved by **0.0e+00**. That one measurement is the cleanest possible demonstration of rotational ambiguity — learn it.

Likely viva questions

1. State precisely how factor analysis differs from PCA.
2. Prove that an orthogonal rotation of W leaves the likelihood unchanged.
3. Why is the free parameter count d + Ld - L(L-1)/2?
4. What is a communality, and what does a low one tell you?
5. Why does FA need Psi to be diagonal?

Evaluating the output of clustering methods [Unit 3]

Core idea. Clustering is unsupervised, so there is no accuracy. You must judge either INTERNALLY from the geometry of the result alone, or RELATIVELY by asking whether two methods agree.

```
silhouette  s = (b - a) / max(a, b)      in [-1, 1]

  a = mean distance to own cluster

  b = mean distance to the NEAREST OTHER cluster

Davies-Bouldin : mean worst-case

  (spread_i + spread_j) / separation_ij     LOWER better

Calinski-Harabasz : between/within dispersion  HIGHER better

ARI : Rand index corrected for chance agreement
```

- Silhouette near 1 means the point is well inside its cluster, near 0 means it is on a boundary, and **negative means it is probably in the wrong cluster**.

- A singleton cluster gets silhouette 0 by convention — the mean distance over zero other members is undefined.
- **ARI is the one to understand properly.** It is immune to cluster relabelling, which is what makes comparing two methods possible at all: cluster 0 from k-means and cluster 2 from Ward may be the same group.
- 'Adjusted' means the agreement expected by chance is subtracted. Without it, splitting data into many tiny clusters inflates the raw Rand index towards 1.
- **The caveat that earns marks:** silhouette and Calinski-Harabasz both reward compact spherical clusters — exactly what k-means produces by construction. Using them alone to declare k-means the winner is close to circular.

> ***In our project***
> All four indices match scikit-learn to 1e-9. On the SKU profiles, k-means and Ward agreed strongly (ARI 0.839) while the GMM disagreed more (0.596) — the expected consequence of a soft, covariance-aware model being allowed elliptical clusters.

Likely viva questions

1. What does a negative silhouette coefficient mean?
2. Why must ARI be invariant to label permutation?
3. Why is it circular to use silhouette to prove k-means beat a density-based method?
4. Two clusterings have ARI 0. What does that tell you?

The Viterbi algorithm [Unit 4]

Core idea. Finds the single most probable hidden-state SEQUENCE using dynamic programming and backpointers. It is the forward algorithm with exactly one operator changed: the sum becomes a maximum.

```
delta_1(j) = pi_j b_j(x_1)

delta_t(j) = b_j(x_t) MAX_i [ delta_t-1(i) a_ij ]

psi_t(j)   = argmax_i [ delta_t-1(i) a_ij ]

z*_T = argmax_i delta_T(i);   z*_t = psi_t+1(z*_t+1)

log form: v_t(j) = log b_j(x_t) + max_i[v_t-1(i) + log a_ij]
```

- Forward SUMS over all paths into state j (total likelihood); Viterbi keeps only the BEST path into each state.
- **Add the emission term AFTER the max.** It does not depend on the predecessor i so it cannot change which i wins — but forgetting it entirely is the classic bug.
- Store the backpointers psi if you need the actual path, not just its score.
- Work in logs: products become sums while max stays max, so logs are an exact reformulation, not an approximation. No scaling constants needed.
- Complexity O(T N^2) in time, O(T N) for the backpointers.
- **Viterbi vs posterior decoding** — the examinable distinction. Posterior decoding takes argmax of gamma_t independently at each t. It maximises the number of individually correct states but **never consults A**, so it can emit a sequence the model assigns probability zero. Viterbi maximises the probability of the path as a whole and is always legal.
- Max-product is the same algorithm in the belief-propagation framework.

In our project

Verified against brute-force scoring of all 2,187 paths: identical path and log-probability. On the real series the two decoders disagree on 21 of 604 days and Viterbi's path is 21.6 log-units more probable; posterior decoding also switches regime 12 times against 8. A constructed test case shows posterior decoding emitting a path of probability exactly zero.

Likely viva questions

1. How does Viterbi differ from the forward algorithm?
2. Why can posterior decoding produce an impossible sequence?
3. Why does Viterbi not need scaling factors?
4. What do the backpointers store and why?

The sum–product algorithm [Unit 4]

Core idea. Exact marginalisation on tree-structured factor graphs by passing local messages. It is the GENERAL algorithm of which forward-backward is a special case, and it unifies most of the dynamic programming in this unit.

```
f(x_1..x_n) = PROD_a f_a(x_a)

var -> factor:  m_{x->f}(x) = PROD_{h in N(x)\{f}} m_{h->x}(x)

factor -> var:  m_{f->x}(x) =

        SUM_{X_f \ x} f(X_f) PROD_{y in N(f)\{x}} m_{y->f}(y)

belief:  b(x) PROPTO PROD_{f in N(x)} m_{f->x}(x)
```

- A factor graph has variable nodes and factor nodes; the joint factorises into local functions over subsets of variables.
- A variable multiplies everything it has heard from elsewhere; a factor multiplies in its own local function and SUMS OUT the variables it is not addressing.
- **On a tree, one inward and one outward sweep gives EXACT marginals.** On a loopy graph, iterating the same updates is loopy belief propagation: approximate, and may not converge.
- **On a chain, the left-to-right messages ARE alpha and the right-to-left messages ARE beta.** Forward-backward is sum-product on an HMM chain.
- **The semiring view:** replace sum with max and the same code computes max-marginals — that is max-product, and tracing its argmaxes gives Viterbi. Sum-product gives marginals; max-product gives MAP.
- Normalise beliefs before interpreting them as probabilities, and normalise messages at each step to avoid underflow.

In our project

The claim was turned into a measurement rather than asserted. Implementing the general message-passing rules and comparing: forward messages vs alpha, backward vs beta, beliefs vs gamma, pairwise beliefs vs xi, and log Z vs the forward log-likelihood — **every difference is exactly 0.0**. Swapping sum for max reproduces the Viterbi path exactly.

Likely viva questions

1. Write both message-passing rules.
2. Why are sum-product marginals exact on a tree but not on a loopy graph?
3. Show how forward-backward is a special case of sum-product.

4. What changes if you replace sum with max?

Linear dynamical systems and the Kalman filter [Unit 4]

Core idea. An LDS is the HMM with a CONTINUOUS hidden state: a vector evolving linearly with Gaussian noise, generating linear noisy observations. The Kalman filter is exact inference for this model — the continuous analogue of forward-backward.

```
z_t = A z_t-1 + w_t,   w ~ N(0, Q)     state
x_t = C z_t   + v_t,   v ~ N(0, R)     observation
predict: mu- = A mu,   P- = A P A^T + Q
gain:    K = P- C^T (C P- C^T + R)^-1
update:  mu = mu- + K(x_t - C mu-),  P = (I - K C) P-
```

- **The Kalman gain K is the whole story.** It weighs model against measurement: small R (trustworthy sensor) drives K up and the estimate follows the data; small Q (smooth dynamics) drives K down and the estimate follows the model.
- The **innovation** x_t - C mu- is the part of today's observation the model did not anticipate — the only part carrying new information.
- After an update the covariance usually DECREASES: measurement reduces uncertainty.
- **Filtering vs smoothing again:** the filter gives p(z_t | x_1:t); the Rauch-Tung-Striebel smoother runs backwards to give p(z_t | x_1:T). Smoothed variance can never exceed filtered variance — it conditions on more data.
- Special cases: A = I is a random-walk state model. Nonlinear systems need EKF, UKF or particle filters.
- The scalar local-level model (A = C = 1) is EXACTLY equivalent to simple exponential smoothing, with alpha equal to the steady-state gain. So fitting q and r by maximum likelihood is a principled way of choosing a smoothing constant instead of picking 0.3 because it looked reasonable.
- For a random walk the point forecast is FLAT while the variance grows linearly — an honest statement that the model knows of no trend, not a failure.

In our project

Fitted by maximum likelihood: q = 0.00273, r = 0.33101, giving a steady-state gain of **0.087** — the model's judgement that most daily movement is noise. On held-out data it beat a naive random walk by **25.2%** on one-step RMSE, and smoothing cut the mean level variance roughly in half.

Likely viva questions

1. What does the Kalman gain balance, and what happens as R -> 0?
2. Why can the smoothed variance never exceed the filtered variance?
3. How is an LDS related to an HMM?
4. Why is a flat forecast with a widening band the correct output for a random walk?
5. What is the innovation and why does it matter?

Quick reference — the whole syllabus on two pages

Unit 3

Topic	One-line summary	The thing people get wrong
Dissimilarity	The metric IS the model assumption	Forgetting to standardise first
K-means	Minimise within-cluster variance; hard assignment	Thinking the lowest inertia gives the best k
GMM	Soft clustering; responsibilities sum to 1	Forgetting a covariance has d(d+1)/2 free entries
EM	E: posterior over z. M: maximise expected complete LL	Not knowing WHY the likelihood cannot decrease
Alt. view of EM	Coordinate ascent on F = log p(x) - KL	Not knowing the bound is tight exactly at the posterior
Factor analysis	cov(x) = WW^T + Psi; shared vs total variance	Treating the loadings as unique
PCA	Orthogonal directions of maximum variance	Applying the Kaiser rule to unstandardised data
Latent dims	Kaiser / variance threshold / scree elbow	Quoting only the rule that gave a convenient answer
Hierarchical	Merge the closest pair; cut the tree afterwards	Assuming high cophenetic r means a good partition
Evaluation	Silhouette, DB, CH internal; ARI relative	Using silhouette to 'prove' k-means wins

Unit 4

Topic	One-line summary	The thing people get wrong
Sequential data	Order carries information; chain rule is exact	Shuffling, or normalising with full-series statistics
Markov models	History compressed into the current state	Rows of A not summing to 1; zero counts left as impossible
HMM	Hidden Markov states, observed emissions	Mixing up z and x
ML / EM	Baum-Welch: normalise expected counts	Using hard counts; uniform init (a saddle point)
Forward-backward	alpha from the past, beta from the future	Confusing filtering with smoothing
Scaling factors	Normalise alpha, keep c_t; log p = sum log c_t	Mixing scaled and unscaled conventions
Viterbi	Forward with max instead of sum, plus backpointers	Dropping the emission term after the max
Sum-product	Message passing; exact on trees	Not seeing that forward-backward IS sum-product
LDS / Kalman	Continuous hidden state; gain balances model vs data	Expecting a random walk to forecast a trend

If you only memorise one table

Evaluation -> Forward. Decoding -> Viterbi. Learning -> Baum-Welch / EM.

Sum-product gives marginals; max-product gives MAP. Filtering uses x_1:t; smoothing uses x_1:T. Complexity is O(T N^2), never N^T.